

EcoRatings Frontend Developer - What to Learn

Overview

This is one of the closest matches to your actual stack so far — React hooks, TypeScript, Tailwind, Redux/Context, and even RAG/LLM visualization line up directly with Echo and Vibe. The gaps below are small, mostly naming/tooling differences from what you already do.

MUST learn

Redux Toolkit (RTK) — vs. the plain Redux you already use

- What it is: the official, modern way to write Redux — `configureStore`, `createSlice` (bundles action types + reducers + action creators in one function), and built-in Immer (write "mutating" code that's actually immutable under the hood).
- Why the JD wants it: RTK is now the standard way Redux is written in production — your Clink AI store is likely already RTK-based or very close to it; the JD just names it explicitly.
- The 20%: if your Clink AI store uses `createSlice`, you already know RTK — just use the term "Redux Toolkit" explicitly. If it's older-style Redux (separate action types/creators/reducers), spend 20 minutes converting one slice mentally: `createSlice` replaces that boilerplate with one object.
- Bridge: same store/action/reducer/selector mental model you already have — RTK just removes the boilerplate.

GOOD to learn

Vite — as a named build tool alongside Webpack

- What it is: a modern, fast dev-server + bundler — uses native ES modules in dev (near-instant startup, no bundling until you build) and Rollup under the hood for production builds. Faster dev experience than Webpack, especially on large codebases.
- Why the JD wants it: listed alongside Webpack as an acceptable build tool — teams are increasingly standardizing on Vite for new projects.
- The 20%: if you haven't used Vite directly (Next.js/Expo abstract this away), spin up one `npm create vite@latest` React+TS project once to see the dev-server speed and config shape (`vite.config.ts` vs. `webpack.config.js`) — that's enough to speak to it in an interview.

Tree shaking & memoization — as explicit technique names

- What it is: tree shaking = bundlers removing unused exports from your final bundle automatically (mostly automatic with ES modules + modern bundlers). Memoization = caching a computed value/component render so it isn't redone unnecessarily (`React.memo`, `useMemo`, `useCallback`).
- Why the JD wants it: named explicitly as a required "understanding" — performance-conscious frontend hiring bar.
- The 20%: you likely already avoid unnecessary re-renders informally — name the specific hooks (`useMemo` for expensive computed values, `useCallback` for stable function references passed to memoized children, `React.memo` to skip re-rendering a component when props haven't changed) so you can speak to them precisely rather than just "I optimize performance."

Radix UI — alongside Shadcn (which is built on top of it) and Material UI

- What it is: unstyled, accessible component primitives (dialogs, dropdowns, tooltips) that handle keyboard nav/ARIA/focus management for you — Shadcn UI is themed, copy-paste components built ON TOP of Radix primitives, so if you've used Shadcn (per your Pixxel project), you've already used Radix indirectly.
- Why the JD wants it: named as one of three acceptable UI library options.

- The 20%: just know the relationship — Radix = unstyled accessible primitives, Shadcn = Radix + Tailwind styling + copy-paste ownership model, Material UI = fully-styled Google Material Design components (a different philosophy, pre-styled and themeable via a theme object).

Not worth deep prep

Everything else in the JD (React hooks, TypeScript, Tailwind CSS, Context API, Git, cross-browser debugging, lazy loading/code splitting) is already genuine, verified experience from your Clink AI work and the Echo/Vibe/Pixxel projects — no new prep needed, just make sure these terms show up explicitly in your resume/interview answers (they already do on this variant).